Les activités

Imene Sghaier

PLAN DU CHAPITRE

- Activity
- Cycle de vie d'une Activity
- Le fichier activity_main.xml
- La classe R

ACTIVITY

- Une activité est la composante principale pour une application Android. Elle représente l'implémentation métier dans une application Android, permettant de gérer l'ensemble des vues et ressources.
- Une activité peut être avec ou sans interface utilisateur. Il est possible d'avoir plusieurs activités dans le même programme. Elle doit toujours être déclarée dans le fichier *Manifest.xml*.
- Une activité n'est pas linéaire, elle est soumise à plusieurs évènements. Chaque événement est représenté dans une méthode.

ACTIVITY

- **Exemple**

```
package com.isetn.first;

import android.support.v7.app.AppCompatActivity;
import android.os.Bundle;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

ACTIVITY

- Dans l'exemple précédent on importe deux classes qui se trouvent dans deux packages différents :
  - AppCompatActivity
  - Bundle.
- AppCompatActivity est une classe qui décrit une activité supportant une barre d'action.

AppCompatActivity

extends FragmentActivity

implements AppCompatCallback TaskStackBuilder.SupportParentable ActionBarDrawerToggle.DelegateProvider

---

```
java.lang.Object
  ↳ android.content.Context
    ↳ android.content.ContextWrapper
      ↳ android.view.ContextThemeWrapper
        ↳ android.app.Activity
          ↳ android.support.v4.app.FragmentActivity
            ↳ android.support.v7.app.AppCompatActivity
```

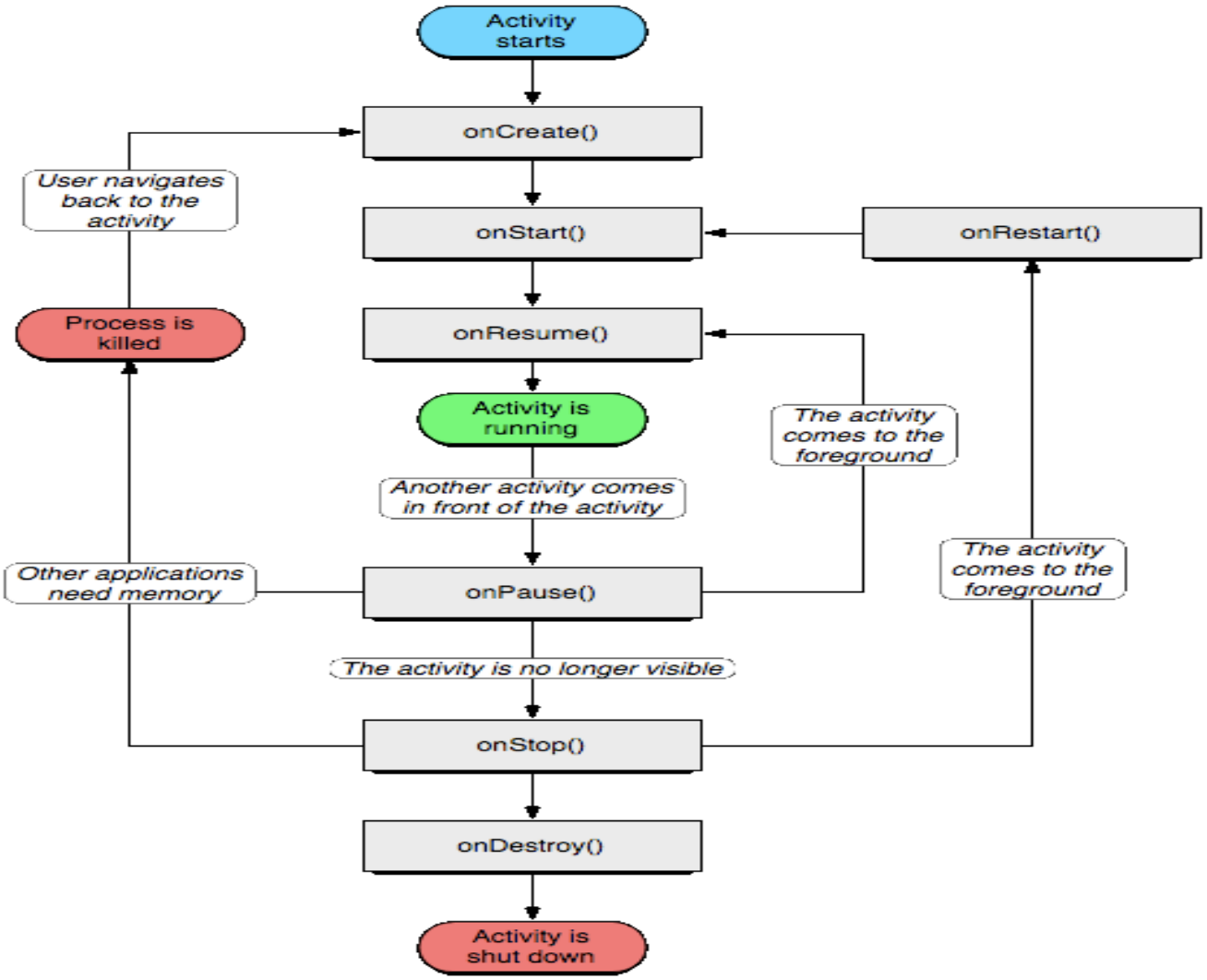
Known Direct Subclasses

ActionBarActivity

ACTIVITY

- **@Override** permet d'indiquer une redéfinition de méthode à partir de la classe mère (ici c'est AppCompatActivity). Nous expliquons ceci dans le TP.
- La méthode onCreate() est la première qui est lancée au démarrage d'une application, mais elle est aussi appelée après qu'une application se soit faite tuée par le système en manque de mémoire!
- **Bundle** est utile lorsque une activité est détruite puis recréée (voir le cycle de vie d'une activité).
- C'est à cela que sert l'attribut de type Bundle :
  - S'il s'agit du premier lancement de l'application ou d'un démarrage alors qu'elle avait été quittée normalement, il vaut **null**.
  - Mais s'il s'agit d'un retour à l'application après qu'elle a perdu le focus et a redémarré, alors il pourra contenir un état sauvegardé de l'application que vous aurez pris soin de constituer. (par exemple : une chanson qui reprend à l'endroit où elle s'est arrêtée avant d'être interrompue par un appel).
- L'instruction **setContentView(R.layout.*activity_main*);** permet d'affecter l'interface graphique à l'activité (les rattacher) et qui est décrite dans le fichier activity_main.xml.

Cycle de vie d'une activity



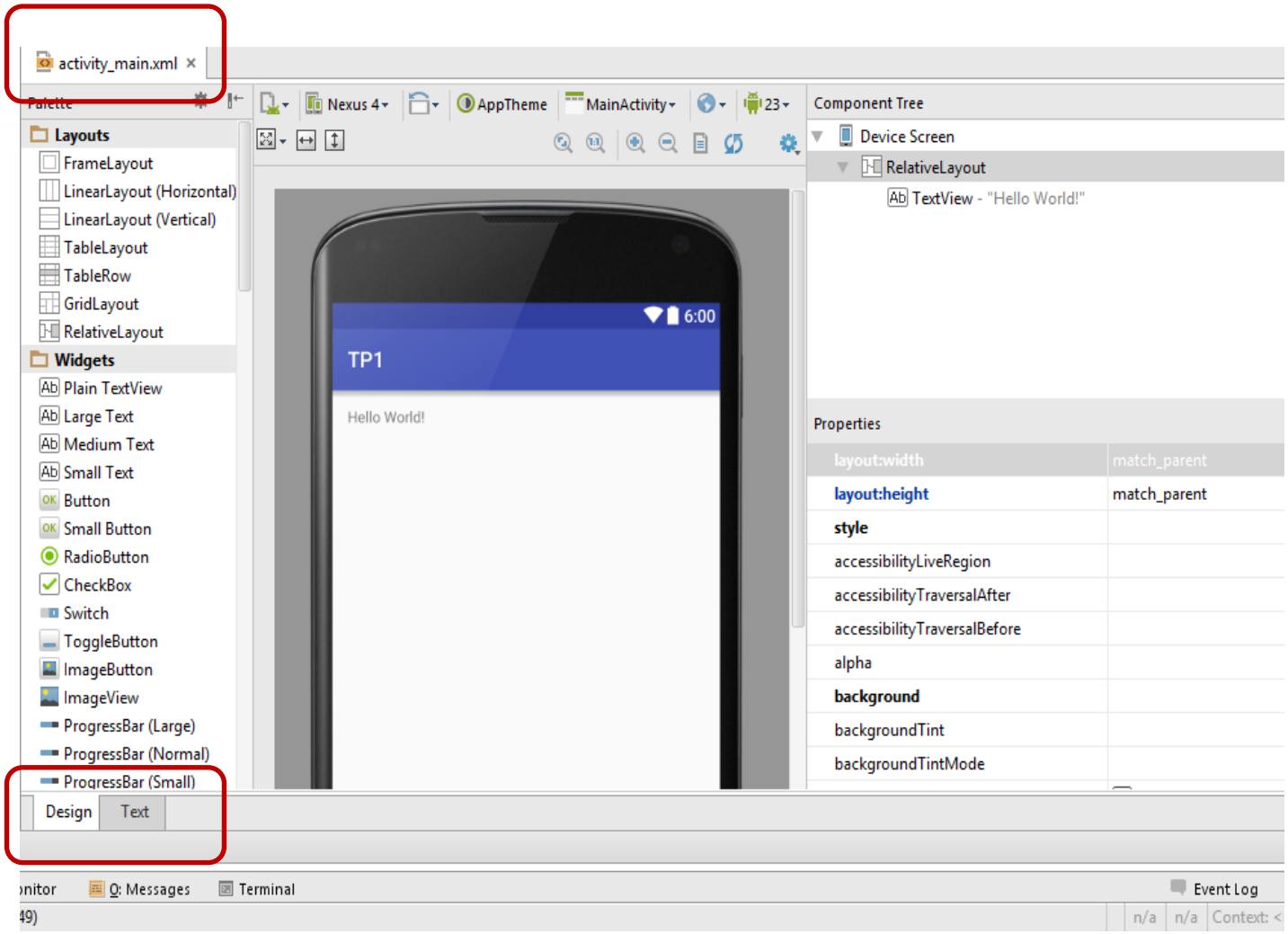
Cycle de vie d'une activity

- **onCreate()** : Cette méthode est appelée à la création d'une activité. Elle permet de l'initialiser. C'est ici que l'interface graphique est spécifiée.
- **onStart()** : Cette méthode est appelée quand l'application est démarrée
- **onResume()** : Cette méthode est appelée quand l'application passe (ou repasse) en avant----plan.
- **onPause()** : Appelée quand l'application passe en arriere----plan et qu'une autre application se met devant.
- **onStop()** : Appelée quand l'application n'est plus visible.
- **onRestart()** : Appelée quand l'application redevient visible.
- **onDestroy()** : Appelée quand votre application

LE FICHIER ACTIVITY_MAIN.XML

- Le fichier **activity_main.xml** généré est repéré par la constante R.layout.activity_main, argument de setContentView() dans l'activité. Son contenu peut être visualisé en mode Design ou Text (XML).
- Ce fichier contient la description de l'IHM.
- Souvent, l'IHM est construite par glisser-déposer sous l'onglet Design qui suggère :
  - les composants graphiques (Widgets)
  - les positionnements (Layouts)

LE FICHIER ACTIVITY_MAIN.XML



La classe R

- **R** est une **classe** <u>générée</u> et <u>mise à jour</u> automatiquement par l'IDE utilisé (dans un fichier R.java). Elle se trouve sous le dossier **app/build/generated** mais qui n'apparait pas dans l'arborescence de l'IDE.
- Le fichier **R.java** définit une classe **R** dans laquelle sont définis les identifiants des ressources de l'application.
- A chaque fois qu'on rajoute une ressource à l'application, un identifiant est généré automatiquement dans cette classe pour permettre par la suite de pouvoir le référencer pour l'utiliser dans le code des activitys de l'application.
- La classe **R** est final donc elle <u>n'est pas héritable</u>.

La classe R

- A l'intérieur de la classe **R** on trouve des plusieurs classes internes, statiques et publiques définies, telles que **string**, **drawable**, **layout**, **menu**, **id**, etc.
- Toutes les classes internes dans **R** sont publiques, donc accessibles depuis l'extérieur, mais comme elles sont membres de la classe **R**, pour y accéder, il faut passer par **R**.
- On utilisera des notations telles que **R.string** puisque ces classes sont statiques.
- Les identifiants sont générés automatiquement dans la classe **R** au moment où on les crée.
- Quand, par exemple nous créons un champ de saisie (Editext) avec comme identifiant prénom, un attribut constant **public static final prénom** est défini automatiquement dans la classe **id** et une valeur lui est attribuée. C'est un numéro unique qui identifie l'élément EditText que nous avons rajouté.
- Cet identifiant nous permettra de manipuler l'élément à partir du code java avec **R.id.prenom**.

La classe R

- Localisez les constantes suivantes dans R.java :
  - activity_main (R.layout.activity_main)
  - app_name (R.string.app_name)

```
/* AUTO-GENERATED FILE.  DO NOT MODIFY.
 *
 * This class was automatically generated by the
 * aapt tool from the resource data it found.  It
 * should not be modified by hand.
 */

package jmf.com.myapplication;
public final class R {
    public static final class layout {
            public static final int activity_main=0x7f040019;
            ...
    }
     public static final class string {
        public static final int app_name=0x7f060014;
     ...
    }
...
}
```

  - R.layout.activity_main indique le fichier activity_main.xml se trouvant dans le répertoire **res/layout**.

La classe R

- Il y a correspondance entre les noms dans le fichier **R.java** et les noms utilisés dans le **programme Android**. Les identificateurs dans **R.java** font référence à des fichiers se trouvant dans le répertoire **res**.
- Par exemple :
- - les images sont accessibles par R.drawable.*nomImage* et correspondent à des fichiers images dans le répertoire **res/drawable**.
- - Les noms sont accessibles par R.string.*nom* et correspondent à des chaînes de caractères dans le fichier **res/values/strings.xml**. On récupère ces noms dans le code source par getResources().getString(R.string.*nom*);

La classe R

- Les composants graphiques ont un identifiant dans le fichier xml qui les contient (par exemple le activity_main.xml).
- Cet identifiant est la valeur de l'attribut android:id et est de la forme "@+id/IditificateurAttribut". On récupère, dans le code, le composant graphique grâce à cet identifiant par l'appel findViewById(R.id. IditificateurAttribut);
- **Exemple : Accès à un composant bouton**

Button leBouton = (Button)findViewById(R.id.leId);

Références

- http://developer.android.com/guide/basics/what-is-android.html
- Cours de Jean-Marc Farinone, maître de conférences au Conservatoire National des Arts et Métiers (CNAM) de Paris
- Cours "Développement sous Android" de Jean-François Lalande, INSA, Institut National des Sciences Appliquées, Centre VAL DE LOIRE